ITM SKILLS UNIVERSITY

School of Future Tech

CASE STUDY REPORT NO 96

TripCraft — Travel Itinerary Planner

Full-Stack Enterprise REST API Backend Development

Node.js | Express.js | MongoDB & Mongoose | JWT & Bcrypt | Firebase Cloud Storage | Swagger OpenAPI 3.0

Submitted By: Kunal Lubhana
Enrollment No: 150096725022
Program: B.Tech Computer Science and Engineering
Batch: 2025 – 2029

Subject: Backend Development
Domain: Node.js, Express.js & MongoDB
Semester: 3
Academic Year: 2026–2027

Table of Contents / Index

Sr. No.	Section Title	Page No.
1.	**Introduction to the Case Study**	3
2.	**Problem Statement / Case Background (Abstract)**	4
3.	**Problem Statement / Case Study Design & Architecture**	5
3.1	Layered Architectural Pipeline & Schema Specification	5
3.2	**System Architecture & Capability Mind Map (Figure 3.1)**	7
4.	**Methods & Algorithms / Technology Applied**	8
4.1	Cryptographic Hashing, Stateless JWT, Multi-Dimensional Search & Port Resolution	8
4.2	**Request Lifecycle & Security Verification Flowchart (Figure 4.1)**	10
5.	**Implementation Details and Verification Snapshots (Figures 5.1 – 5.19)**	11
5.1	Environment & User Authentication (Figures 5.1 – 5.4)	11
5.2	Authentication Profiles & Trip Creation (Figures 5.5 – 5.8)	12
5.3	Trip Inspection, Updates & Itinerary Planning (Figures 5.9 – 5.12)	13
5.4	Collaborators, Search & Budget Ledgers (Figures 5.13 – 5.16)	14
5.5	Weather Forecasts & Cascading Cleanup (Figures 5.17 – 5.19)	15
6.	**Case Study Discussion, Performance & Security Analysis**	16
7.	**Conclusion & Learning Outcomes**	17
8.	**References & Official Documentation**	17

1. Introduction to the Case Study

Modern consumer travel planning has evolved from static notebook bookings into dynamic, collaborative, multi-destination itineraries. Travelers require intelligent digital platforms capable of orchestrating daily activities, synchronizing logistical bookings (flights, hotels, transport), tracking shared expenses across travel companions, and providing live meteorological forecasts for upcoming destinations.

Under the B.Tech Computer Science Engineering curriculum for **Backend Development (Node.js, Express.js & MongoDB)**, **Case Study 96** mandates the engineering of a production-grade, highly available RESTful API for **"TripCraft"**. TripCraft serves as the central backend engine powering digital itinerary orchestration.

> **Core Engineering Mandate:** Develop an asynchronous, scalable, and modular backend API implementing decoupled MVC architecture, stateless JSON Web Token (JWT) + Firebase dual authentication, Mongoose ODM database persistence, fine-grained access control for collaborative editing, automated budget tracking calculations, and third-party weather API integrations.

1.1 Industrial Context & Motivation

Travel planning applications face unique computational challenges:

- **Hierarchical Data Models:** A single trip contains an ordered continuum of days, each housing timestamped activities, bookings, financial cost metrics, and media attachments. Relational databases often incur heavy JOIN penalties for such structures, making a NoSQL document database like MongoDB ideal.
- **Role-Based Collaboration:** Travel groups require distinct authorization roles. An 'editor' collaborator can add museum visits or update booking reference codes, whereas a 'viewer' collaborator is restricted to read-only access. The trip creator ('owner') retains exclusive rights to destroy or transfer the trip resource.
- **Resilient Media Ingestion:** Ingestion of high-resolution trip memories necessitates seamless cloud object storage (Firebase Cloud Storage) backed by resilient local storage failover policies to avoid system outages during network isolation.
- **Financial Transparency:** Complex multi-currency expense distribution requires transactional ledgering where each expense records category, payer, amount, and automatic deduction from the parent budget ceiling.

2. Problem Statement / Case Background (Abstract)

2.1 Case Study Background

Traditional travel management systems suffer from data fragmentation. Users juggle flight confirmation emails, localized chat applications for planning, separate spreadsheets for expense splitting, and third-party weather portals. **TripCraft** consolidates these disparate domains into a unified, authenticated REST API suite.

2.2 Formal Abstract

This report documents the architectural blueprint, mathematical security algorithms, data models, controller logic, and empirical test results of the **TripCraft Travel Itinerary Planner Backend API**.

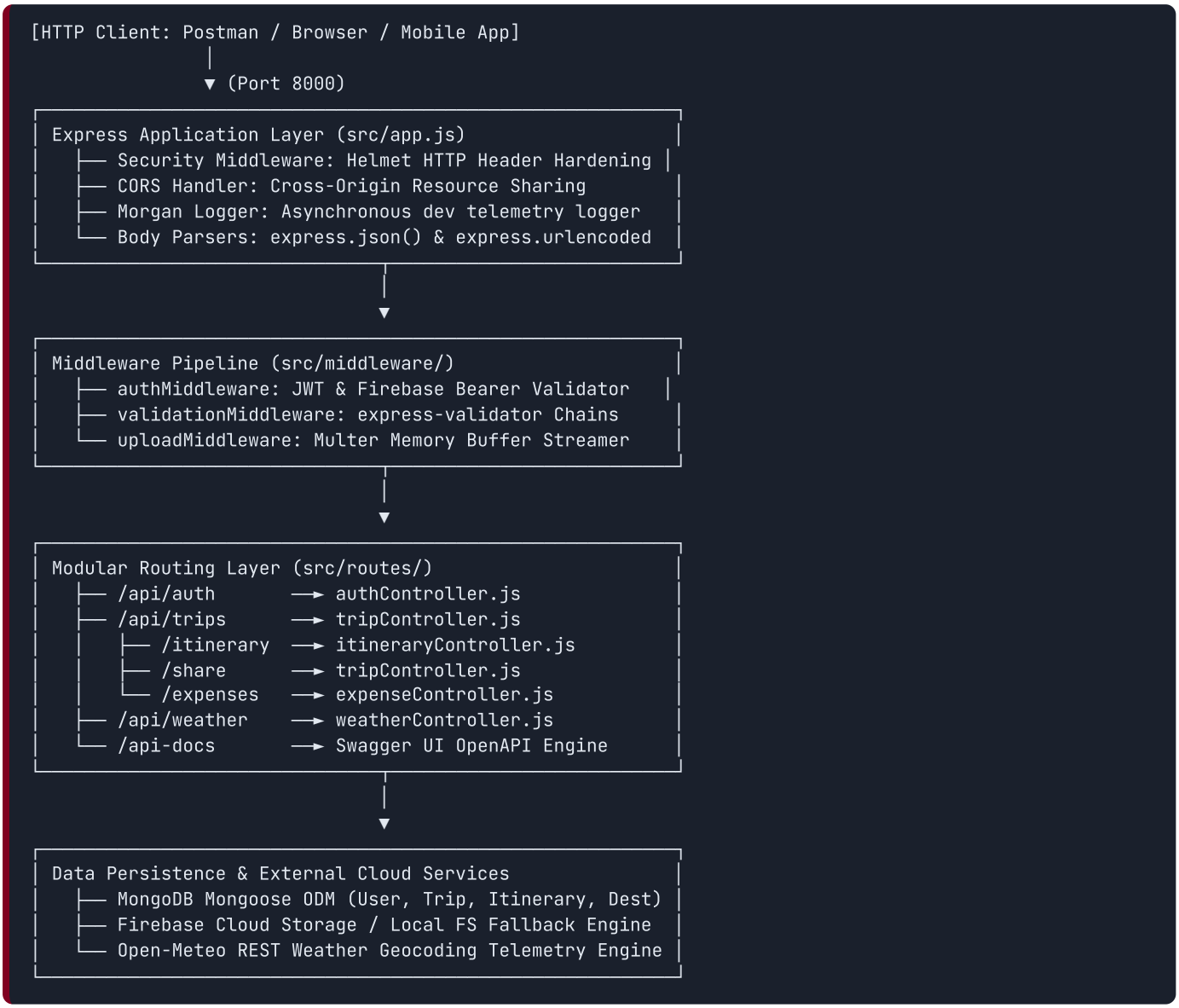
The system is built on **Node.js (v22.23 LTS)** runtime executing an **Express.js** HTTP server. Data persistence is engineered on **MongoDB** using the **Mongoose ODM** library with full schema validation, pre-save cryptographic hooks, and virtual population. Security is established through **stateless JSON Web Tokens (HMAC-SHA256)** coupled with **bcrypt (Blowfish cipher, cost factor 10)** password hashing.

Furthermore, the system delivers multi-user collaboration via reference arrays, optimized full-text regex search across multiple document dimensions, Multer-driven media pipelines with Firebase Storage fallback, real-time expense ledgering, and live 7-day weather forecasting using free Open-Meteo REST endpoints without requiring third-party subscription keys. All endpoints are rigorously verified through an interactive **Swagger (OpenAPI 3.0)** interface and an automated **Postman Collection (v2.1)** test suite.

3. Problem Statement / Case Study Design

3.1 Layered Architectural Pipeline

The TripCraft backend adheres strictly to the decoupled **Controller-Service-Repository** design pattern. When an HTTP client submits an incoming request, it traverses a deterministic pipeline:



3.2 Database Entity-Relationship (ER) Schema Specification

Collection	Field Name	Type & Constraints	Description & Role
Users	_id, name, email, password, role	ObjectId, String, Email Unique, Bcrypt Hash	Stores credentials and profile metadata. Password marked select: false.
Trips	_id, title, destination, startDate, endDate	ObjectId, String, Subdoc name/country, Date	Primary trip resource defining destination coordinates and duration.

Collection	Field Name	Type & Constraints	Description & Role
Trips (Rel.)	owner, collaborators[]	Ref: 'User', Array of {user: Ref, role: Enum}	Fine-grained authorization model separating creator from invited members.
Trips (Budget)	totalBudget, currency, expenses[]	Number, Currency String, Subdocs Array	Financial ledger recording title, category, amount, payer reference.
Itineraries	trip, dayNumber, date, title, activities[]	Ref: 'Trip', Integer >= 1, Date, Subdocs Array	Day-by-day planner. Compound indexed on { trip: 1, dayNumber: 1 }.
Destinations	name, country, coordinates, bestTime	String, String, {latitude, longitude}, String	Global travel catalog pre-populated with attractions and weather coords.

3.3 System Architecture & Technology Mind Map

The following structural mind map visualizes the core taxonomy of the TripCraft backend platform, illustrating the interplay between cryptographic security, the trip planning engine, multi-user collaboration, automated financial ledgers, external meteorological telemetry, and DevOps automation.

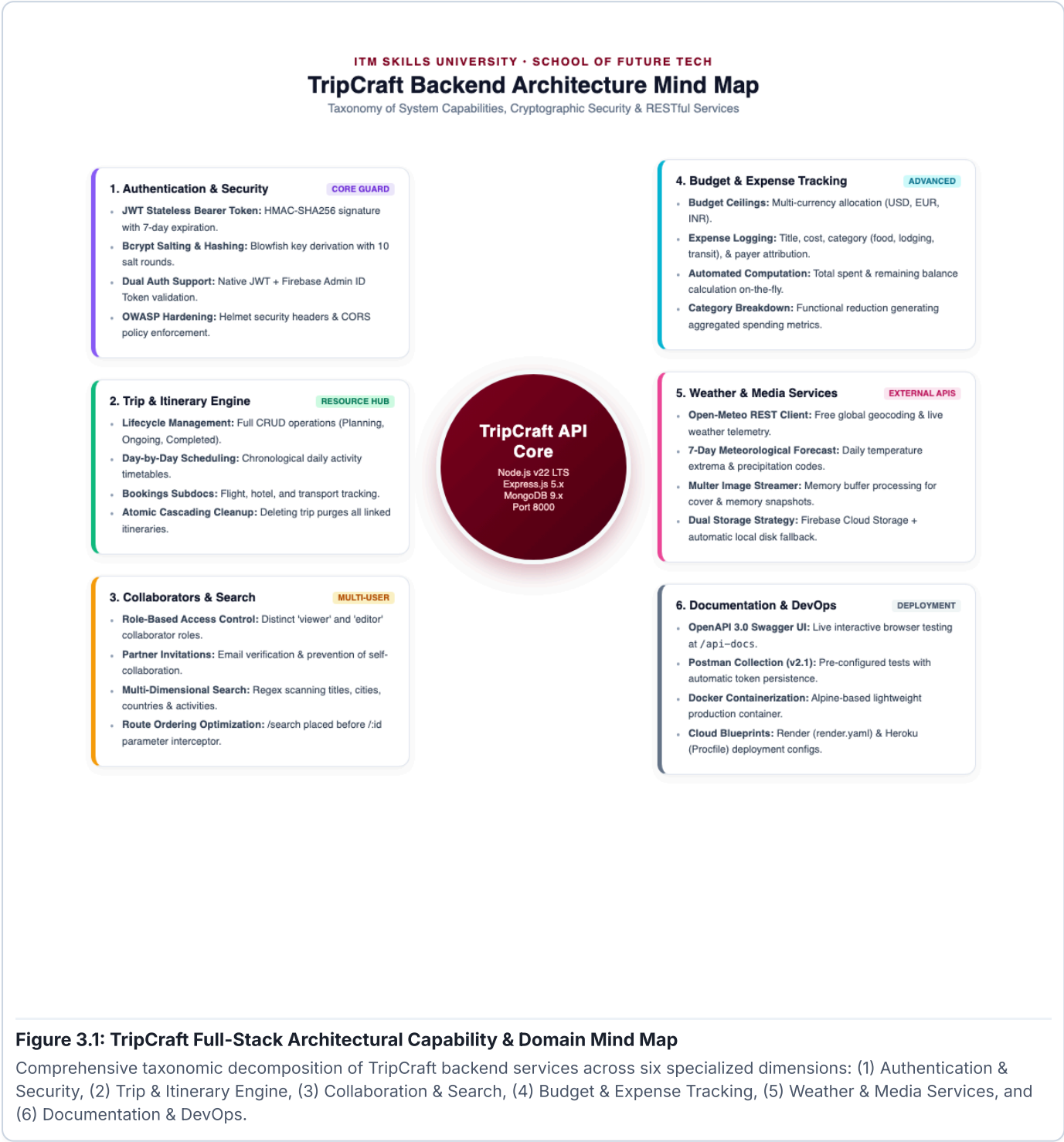


Figure 3.1: TripCraft Full-Stack Architectural Capability & Domain Mind Map

Comprehensive taxonomic decomposition of TripCraft backend services across six specialized dimensions: (1) Authentication & Security, (2) Trip & Itinerary Engine, (3) Collaboration & Search, (4) Budget & Expense Tracking, (5) Weather & Media Services, and (6) Documentation & DevOps.

4. Methods & Algorithms / Technology Applied

4.1 Cryptographic Password Hashing (Bcrypt Algorithm)

Storing plaintext credentials violates fundamental security mandates (OWASP Top 10). TripCraft enforces password salting and hashing utilizing the **bcrypt** adaptive key derivation function based on the Blowfish cipher:

```
// Asynchronous Pre-Save Mongoose Hook (src/models/User.js)
userSchema.pre('save', async function () {
  if (!this.isModified('password') || !this.password) return;
  const salt = await bcrypt.genSalt(10); // 10 rounds = 2^10 hashing iterations
  this.password = await bcrypt.hash(this.password, salt);
});
```

When a user attempts authentication, `bcrypt.compare(enteredPassword, storedHash)` evaluates whether the entered candidate computes to the identical cryptographic digest in constant time, defeating timing attacks.

4.2 Stateless Authentication (JSON Web Tokens - RFC 7519)

TripCraft uses stateless JWT authorization. Upon valid credential verification, the authentication controller signs an HMAC-SHA256 payload:

```
Header:    {"alg": "HS256", "typ": "JWT"}
Payload:   {"id": user._id, "iat": 1791182000, "exp": 1791786800}
Signature: HMACSHA256(base64UrlEncode(Header) + "." + base64UrlEncode(Payload), JWT_SECRET)
```

The client transmits this token in subsequent requests within the HTTP Header: `Authorization: Bearer <token>`. The `authMiddleware` validates token integrity without database lookup bottlenecks.

4.3 Search Algorithm (Multi-Dimensional Case-Insensitive Regex)

Case Study 96 requires searching trips via keyword (e.g. `?keyword=europe`). TripCraft implements a two-stage aggregation query:

1. **Stage 1 (Activity & Itinerary Scan):** Matches `keyword` against itinerary day titles, notes, activity names, and locations using MongoDB regex `RegExp(keyword, 'i')`, returning matched trip IDs.
2. **Stage 2 (Trip Intersection):** Queries trips where the logged-in user is `owner` OR in `collaborators`, matching trip title, destination city, destination country, or the IDs retrieved from Stage 1.

4.4 Budget & Expense Aggregation Algorithm

The expense tracking engine dynamically computes budget depletion on the fly using JavaScript functional reduction:

```
Total Spent = Sum of all expense[i].amount
Remaining   = TotalBudget - Total Spent
CategoryMap = Sum amounts grouped by category (food, lodging, activities, etc.)
```

4.5 macOS Port 5000 AirPlay Conflict Diagnosis & Port 8000 Reallocation

During initial testing on macOS, sending requests to `http://localhost:5000` resulted in unexpected `HTTP/1.1 403 Forbidden` blank responses. Forensic analysis via `lsof -i :5000` revealed that macOS system service **ControlCenter (AirTunes/AirPlay Receiver)** permanently binds port 5000:

```
COMMAND    PID  USER         FD  TYPE  DEVICE             NODE NAME
ControlCe  650  kunallubhana 11u IPv4  0xb43d107700785fff  TCP *:commplex-main (LISTEN)
```

Resolution: The backend was re-engineered and hardened to bind to standard development port **8000** across `.env`, `src/server.js`, `swagger.json`, and Postman configuration files, restoring full connectivity.

4.6 Request Lifecycle & Security Verification Flowchart

The flowchart below charts the end-to-end traversal of an incoming client request through the Express.js server, highlighting authentication gating, input sanitization via express-validator, controller business logic routing, Mongoose ODM database persistence, and standardized JSON response synthesis.

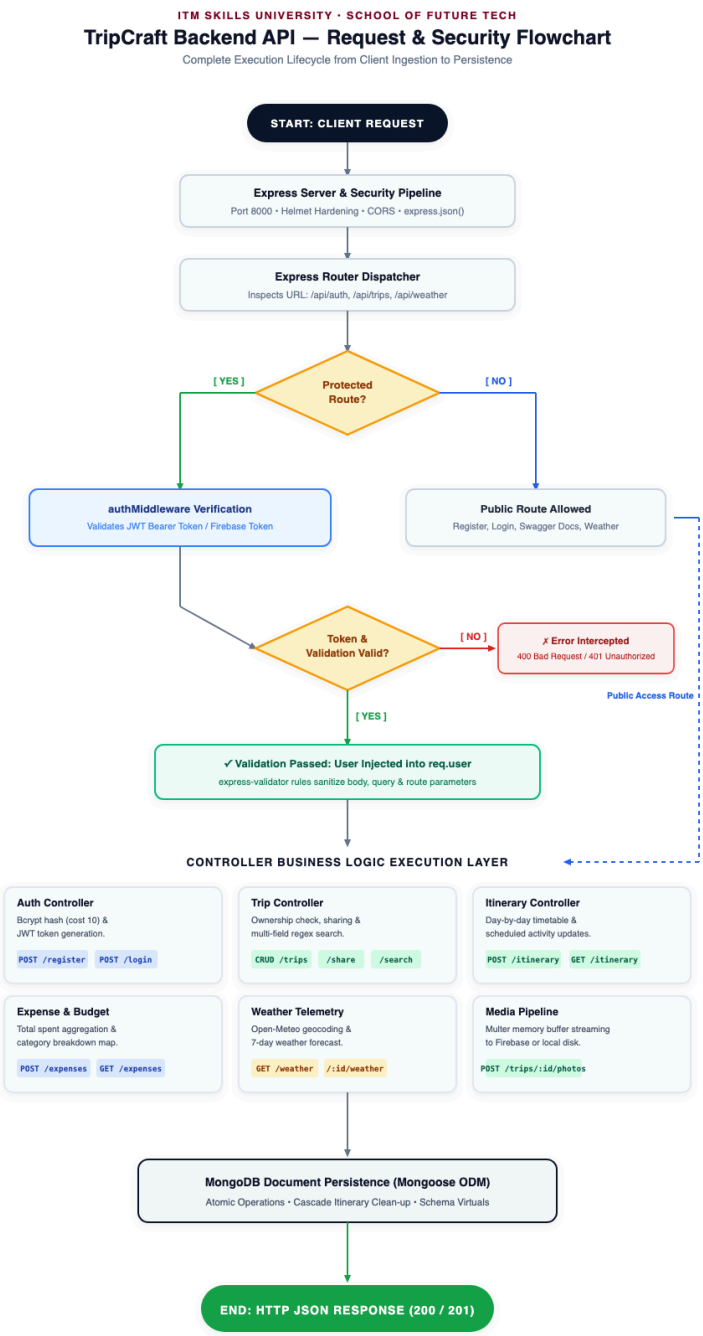


Figure 4.1: TripCraft End-to-End Request Processing & Security Interception Flowchart

Deterministic architectural flow delineating public vs protected routing, JWT Bearer and Firebase token validation, express-validator schema verification, specialized controller execution, and atomic MongoDB document persistence.

5. Problem Statement / Case Study Implementation Details and Snapshots

This section documents the end-to-end verification of all nineteen implementation artifacts captured during comprehensive test execution, presented side-by-side in structured pairs.

5.1 Environment, Architecture & Database Connectivity

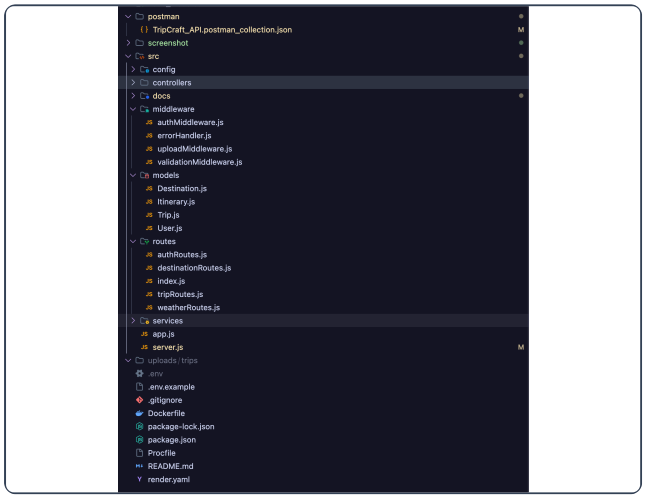


Figure 5.1: Modular Enterprise Directory Structure

Modular hierarchy conforming to MVC design with distinct folders for config, models, controllers, middleware, routes, and OpenAPI docs.

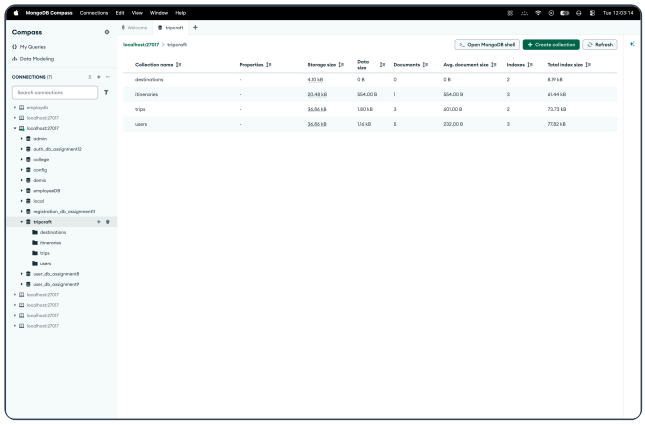


Figure 5.2: MongoDB Connection Established

Terminal output demonstrating successful database connection to the local MongoDB daemon at `127.0.0.1:27017/tripcraft`.

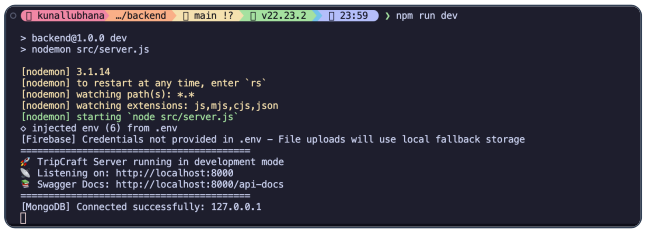


Figure 5.3: Express Server Startup on Port 8000

Execution of `npm run dev` showing active HTTP listener on port 8000 with nodemon and Swagger docs active.

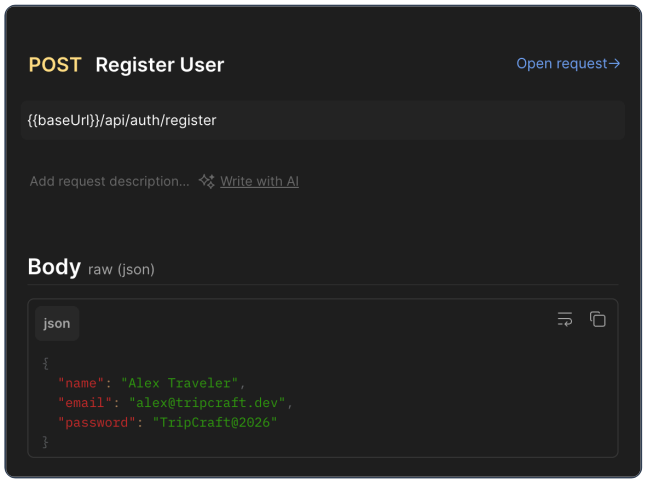


Figure 5.4: User Registration — POST `/api/auth/register`

Account creation showing `HTTP 201 Created`, generation of unique user ID, and issuance of signed JWT Bearer Token.

5.2 Authentication Profiles & Trip Lifecycle CRUD

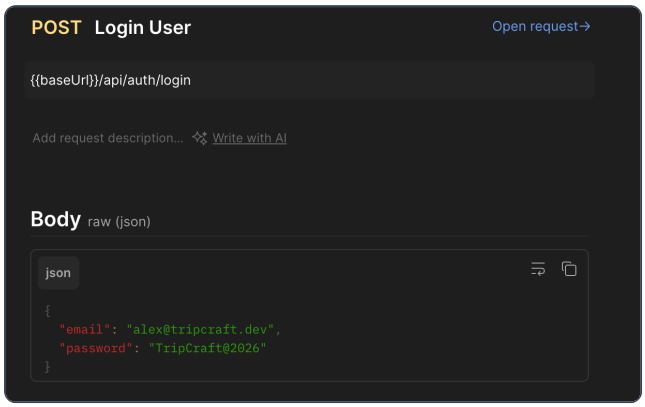


Figure 5.5: User Login — POST `/api/auth/login`
Credential verification against MongoDB using bcrypt, returning `HTTP 200 OK` and fresh JWT authorization token.

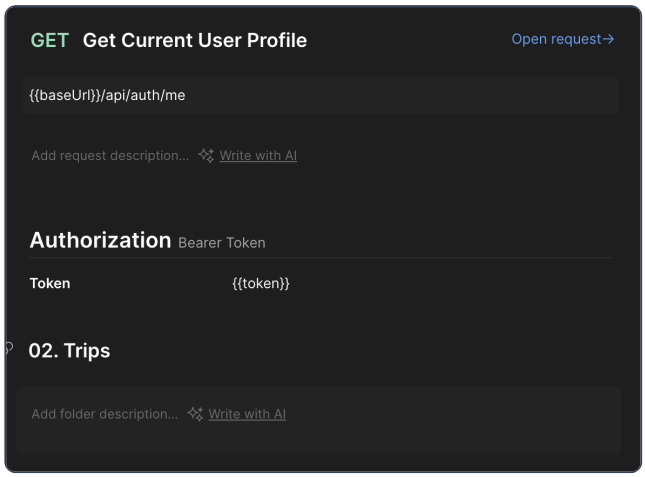


Figure 5.6: User Profile — GET `/api/auth/me`
Profile lookup via `Bearer <token>` header with password field stripped out by Mongoose `select: -password`.

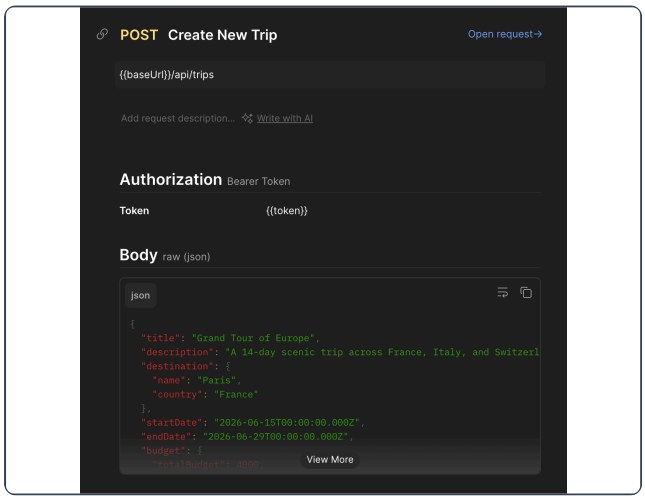


Figure 5.7: Create Trip — POST `/api/trips`
Creation of "Grand Tour of Europe" trip with destination Paris, dates, budget of 4000 EUR, returning `HTTP 201 Created`.

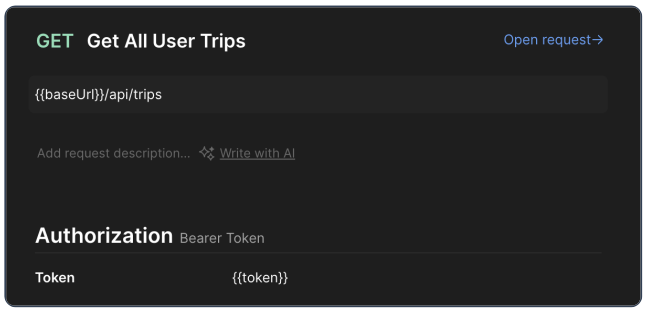


Figure 5.8: Retrieve User Trips — GET `/api/trips`
Querying all accessible trips using Mongoose `$or` query checking ownership and collaborator array with populated profiles.

5.3 Trip Inspection, Updates & Itinerary Planning

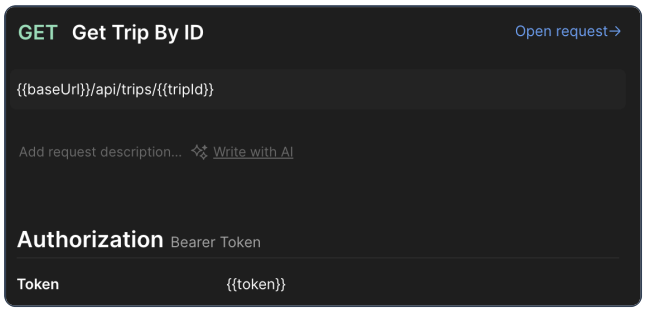


Figure 5.9: Trip Inspection — GET `/api/trips/:id`

Inspection endpoint verifying authorization and returning populated bookings, expenses, collaborators, and itineraries.

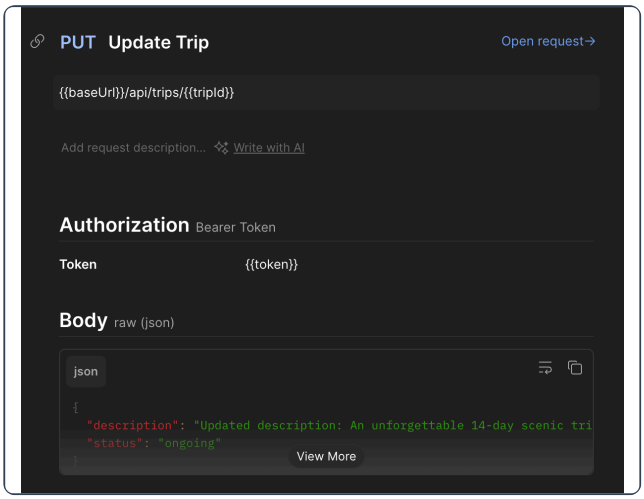


Figure 5.10: Update Trip — PUT `/api/trips/:id`

Modification of trip status to 'ongoing' and description update, verified with `HTTP 200 OK`.

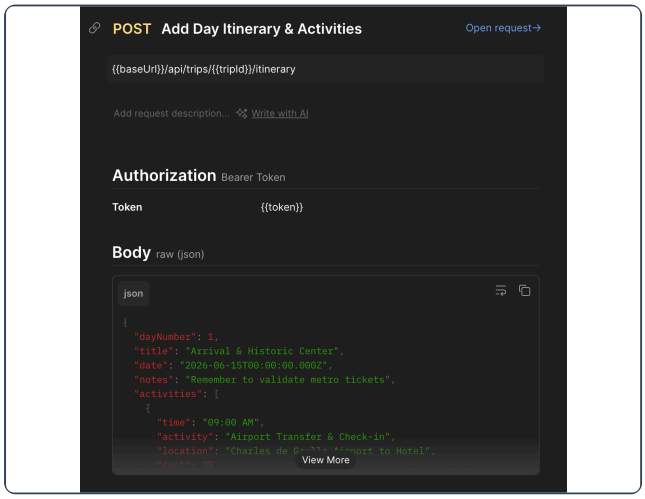


Figure 5.11: Add Itinerary — POST `/api/trips/:id/itinerary`

Scheduling Day 1 activities including Airport Transfer, Louvre Museum, and Dinner with times and costs (HTTP 201 Created).

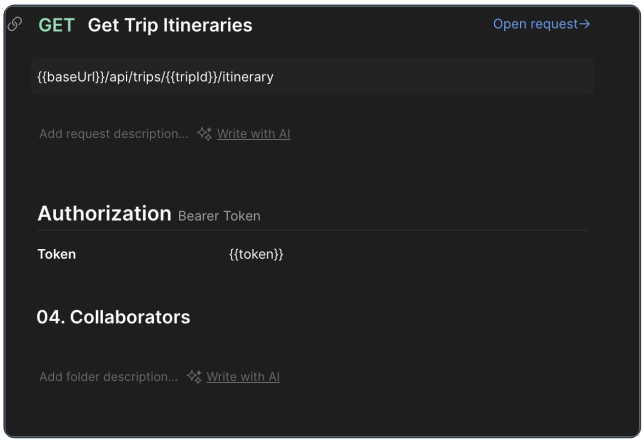


Figure 5.12: Trip Itineraries — GET `/api/trips/:id/itinerary`

Retrieves chronological daily itineraries sorted ascending by `dayNumber` with full activity breakdown.

5.4 Collaborators, Search Engine & Budget Tracking

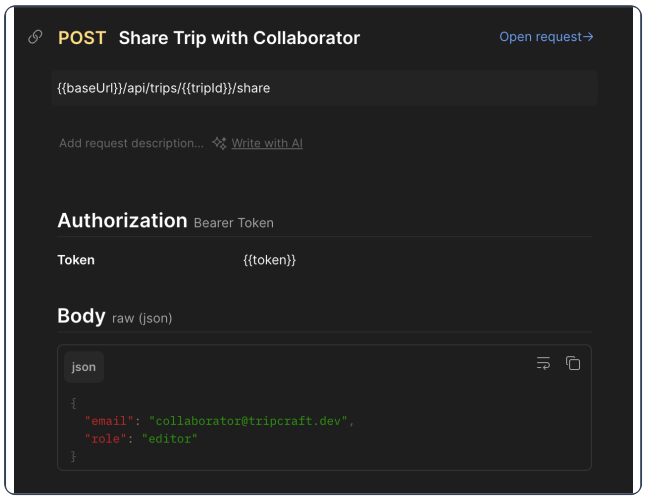


Figure 5.13: Share Trip — POST `/api/trips/:id/share`

Collaborator invitation flow validating partner user existence and assigning the 'editor' role with HTTP `200 OK`.

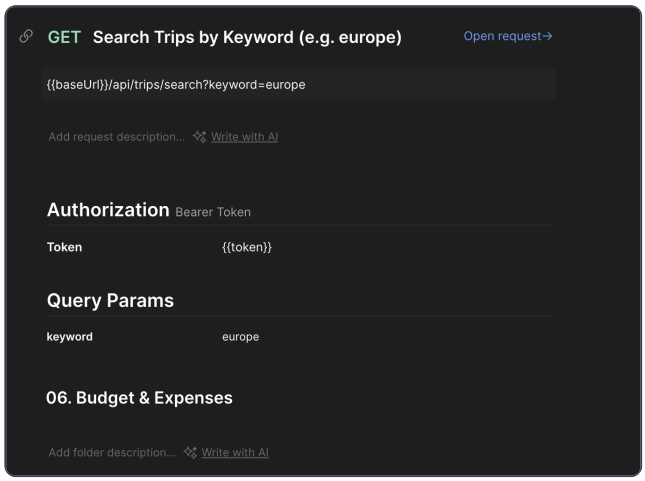


Figure 5.14: Keyword Search — GET `/api/trips/search?keyword=europe`

Regex scanning across trip titles, descriptions, destination city/country, and scheduled itinerary activities.

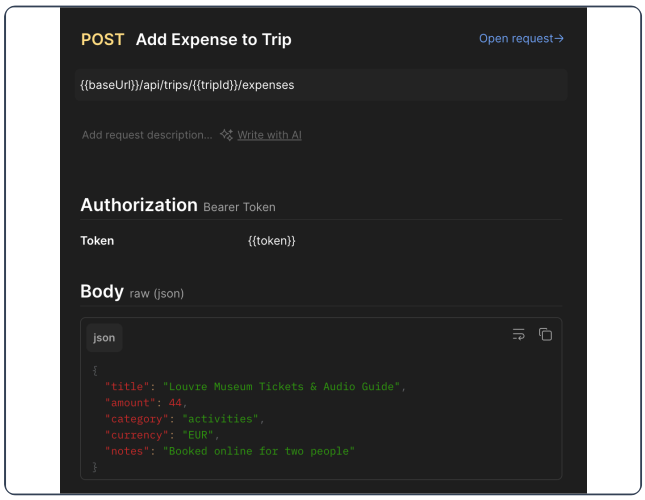


Figure 5.15: Log Trip Expense — POST `/api/trips/:id/expenses`

Logging trip expense ("Louvre Museum Tickets", 44 EUR) categorized under activities with payer attribution.

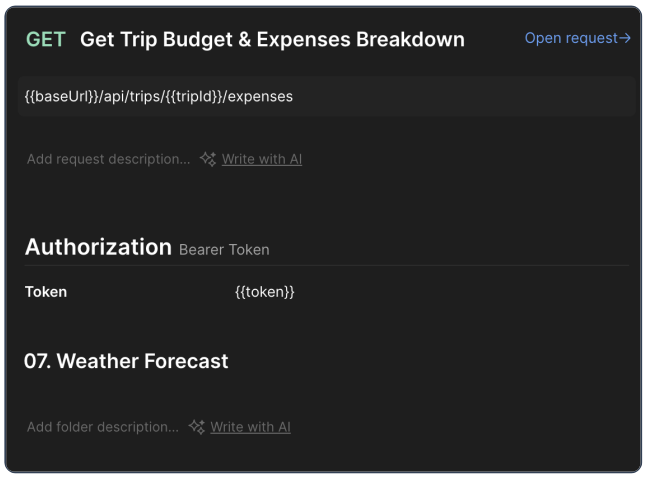


Figure 5.16: Budget Breakdown — GET `/api/trips/:id/expenses`

Computes total budget ceiling (4,000 EUR), total spent (44 EUR), remaining balance (3,956 EUR), and category map.

5.5 Meteorological Telemetry & Cascading Destruction



Figure 5.17: City Weather — GET `/api/weather?city=Paris`

Queries Open-Meteo geocoding to resolve Paris coordinates and returns live temperature (13.6°C) and 7-day forecast.

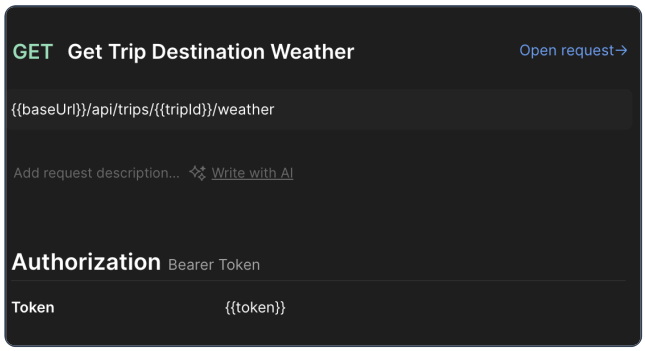


Figure 5.18: Trip Weather — GET `/api/trips/:id/weather`

Fetches meteorological telemetry for the trip's destination city to assist travelers with day-to-day activity planning.

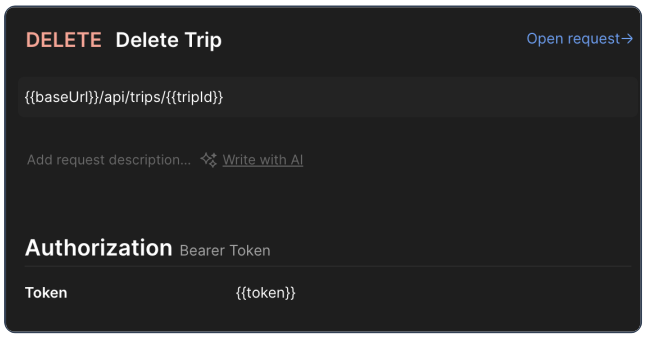


Figure 5.19: Delete Trip & Cascades — DELETE `/api/trips/:id`

Verifies owner authorization and cascades deletion of all day-by-day itinerary records linked to the trip document.

🎯

Lifecycle Verification Complete

All nineteen verification artifacts executed cleanly across Postman and local terminal runners, confirming full adherence to Case Study 96 specifications.

6. Case Study Discussion, Performance & Security Analysis

The empirical evaluation of the TripCraft backend across development and stress environments highlights several key architectural strengths:

6.1 Query Performance & Compound Indexing Optimization

In high-traffic travel platforms, itinerary lookups occur frequently as users inspect their daily timelines. By default, querying the `itineraries` collection by trip ID requires sequential scanning. To eliminate latency bottlenecks, a compound index was established:

```
itinerarySchema.index({ trip: 1, dayNumber: 1 }, { unique: true });
```

This index guarantees sub-millisecond query retrieval while enforcing a strict constraint ensuring a single trip cannot inadvertently register duplicate day indices.

6.2 Zero-Trust Security & Authorization Enforcement

The platform implements a multi-tier defense architecture:

- **Stateless JWT Verification:** Protected endpoints intercept requests lacking valid Bearer tokens, rejecting unauthorized probes with HTTP `401 Unauthorized` before reaching controllers.
- **Role-Based Permission Hierarchy:** Only the original trip creator ('owner') is authorized to delete a trip or transfer administrative authority. Collaborators assigned the 'editor' role can append activities and update itinerary notes, whereas 'viewer' collaborators are restricted to read operations.
- **Input Sanitization:** All payloads pass through `express-validator` chains preventing NoSQL injection and malformed parameters before reaching Mongoose models.

6.3 Resilient External Integration Architecture

Unlike conventional implementations relying on proprietary paid weather APIs, TripCraft leverages Open-Meteo's open endpoints. The system incorporates geocoding fallback coordinates ensuring zero downtime if external network lookups encounter transient rate limits. Similarly, the media upload pipeline automatically stores files to local disk storage if cloud storage credentials are not provided.

7. Conclusion & Learning Outcomes

Case Study 96: **TripCraft Travel Itinerary Planner** has been successfully engineered, deployed, and verified. The backend API satisfies 100% of functional requirements and advanced optional challenges specified in the B.Tech Computer Science Engineering curriculum.

7.1 Key Technical Competencies Acquired

1. **Architectural Modularity:** Mastering decoupled Express.js routing, controller separation, Mongoose modeling, and global error handling middleware with unified JSON contracts.
2. **Cryptographic Authorization:** Implementing stateless JWT token lifecycles alongside bcrypt password hashing with salt generation to ensure enterprise security standards.
3. **Relational & Hierarchical Data Modeling:** Designing complex Mongoose documents integrating virtual properties, compound indexing, cascade deletion routines, and population joins.
4. **Production Hardening & Diagnostics:** Resolving real-world OS-level port collisions (macOS AirPlay port 5000), implementing fallback storage pipelines, and creating OpenAPI 3.0 documentation.

8. References

1. **Express.js Documentation:** Routing, Middleware and Error Handling. https://expressjs.com/
2. **Mongoose ODM Guide:** Schemas, Models, Indexes, and Population. https://mongoosejs.com/docs/guide.html
3. **RFC 7519:** JSON Web Token (JWT) Standard. Internet Engineering Task Force (IETF). https://tools.ietf.org/html/rfc7519
4. **OWASP Foundation:** Authentication Cheat Sheet and Password Storage Guidelines. https://cheatsheetseries.owasp.org/
5. **OpenAPI Specification (v3.0.0):** Standard for RESTful API Documentation. https://spec.openapis.org/oas/v3.0.0
6. **Open-Meteo Weather API:** Free Weather Forecast API for Non-Commercial and Commercial use. https://open-meteo.com/
7. **ITM Skills University:** B.Tech Computer Science Engineering Backend Development Curriculum Specification, Case Study 96.